

AI駆動開発の中での GitHubの活用方法

本Sessionでは我々Sun AsteriskがAI駆動開発のプロジェクトの中でどのようにGitHubを活用しているのかのご紹介をさせていただきます。実装タスクの管理、進捗管理における各種メトリクスの管理など現場で育まれた手法をご紹介します。

 本日のコアテーマ：GitHubを単なるコード管理ではなく“AI駆動開発とマネジメントの共通基盤”にする



Enginner / Division Manager
Creative & Engineering
エンジニア

岡藤 裕太

Yuta Okafuji

Profile

法人営業を経験後、受託開発企業にてエンジニアとしてのキャリアをスタート。

バックエンドからフロントエンド、モバイル、インフラ構築、CMS構築などを担当、コーポレートサイトやECサイト、モバイルアプリケーションの構築を経験する。お客様と直接やりとりをしつつ、見積りや設計、実装、レビュー、テストを担当。

Sun*に入社後、フロントエンド領域のテックリードとして案件に参画し、Frontendチームのマネージャーに就任。その後バックエンドチームのマネージャーに就任。2024年のベストマネージャー賞を受賞。現在は、Division Managerとして、エンジニア組織を統括する。

Works

HR系サービス開発支援

人材紹介サービス

自社開発チームのリードとして、新機能の要件定義・見積り・設計・開発・テスト設計、負債の解消やコーディング規約の作成等を担当。

社内開発プラットフォーム開発支援

英手印刷会社

モダンなアプリケーション開発における技術アドバイザーとして参画。業務アプリを開発するための共通プラットフォームの開発を支援。

要件定義

基本設計

詳細設計

Project
Management

Management
Support

Task Management

Architecture 設計

Infra 構築

Frontend
Engineering

Backend
Engineering

Test 設計

Test 実施

運用設計

運用支援

Release 設計

今日のメッセージ



スピードに同期する 管理の仕組み

AI駆動開発により、実装スピードは劇的に加速します。これに合わせて、設計やタスク管理、承認プロセスなどのSDLC（開発ライフサイクル）全体の運営スピードも同期して変革する必要があります。



一元管理の 中心としてのGitHub

GitHubは単なるソースコードの「保管庫」ではありません。基本設計から詳細設計のIssue化、開発PRへの接続、自動テスト、リリースに至るまで、すべてのコラボレーションを集約する共通基盤として機能します。



可視化なき高速開発の 難しさを解消

開発プロセスが高速化すると、かえって進行中のボトルネックや滞留タスクが見えにくくなります。リアルタイムに可視化（見える化）する仕組みを統合することで、AIの恩恵を最大化しコントロールを維持します。

Sun* AI Ready SDLCとは

「AI駆動開発＝単なるコードの自動生成」ではありません。Sun*が提唱する**AI Ready SDLC**は、要件定義から運用まで、**ソフトウェア開発ライフサイクル全体をAIとSSOT（一元管理）を前提に再構築する包括的アプローチ**です。



アップデート： 非エンジニア（ビジネス側）の参画を容易にするため、最も上流の**要件定義フェーズ**からGitHubを**共通プラットフォーム（SSOT）**として採用。ドキュメントおよび議論をGit管理することで、開発全体の整合性を劇的に向上させています。

要件定義・設計のSSOT (Single Source of Truth) 化

開発の齟齬をなくすために最も重要なのが**SSOT（信頼できる唯一の情報源）**の確立です。Sun*では開発チームだけでなく、非エンジニア（ビジネス側）のメンバーも要件定義・ドキュメント管理にGitHubを利用しています。



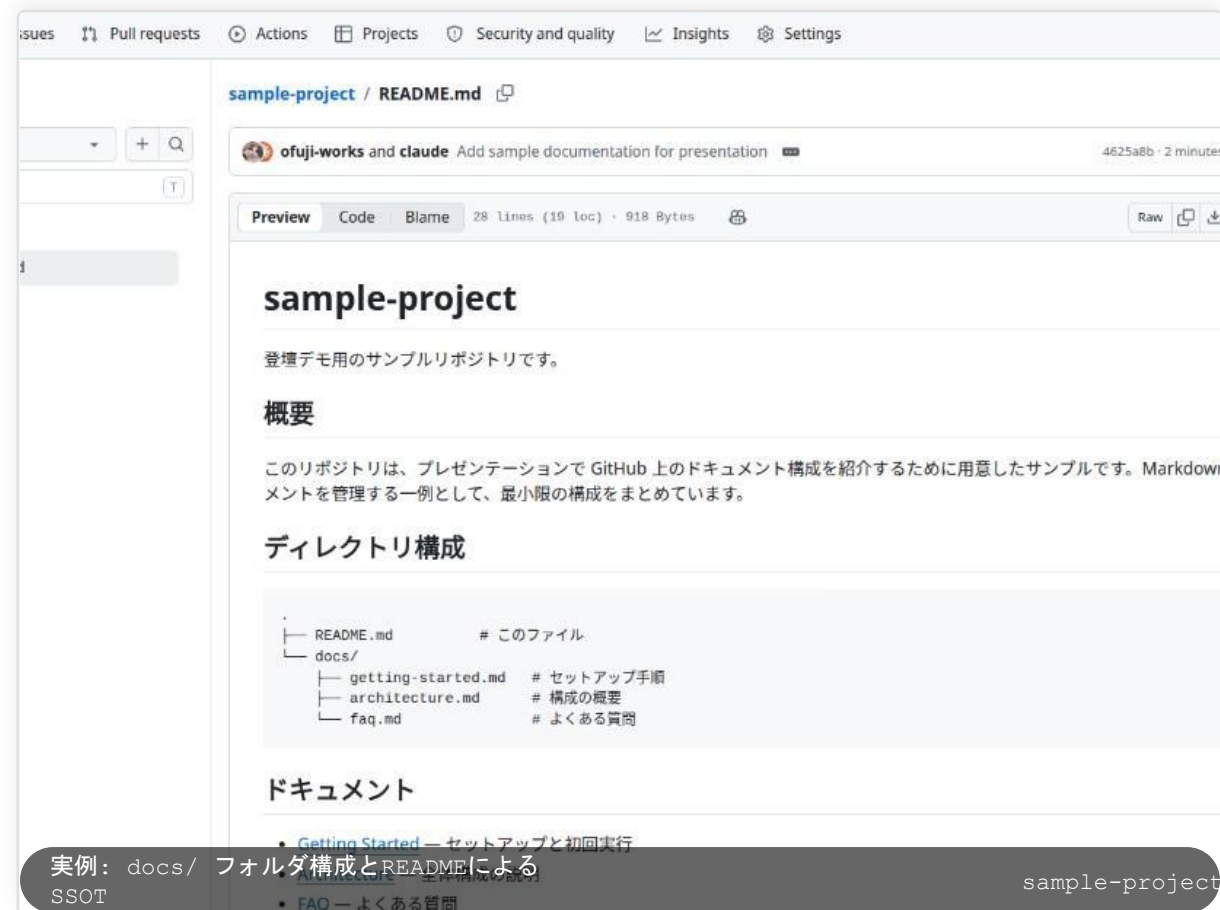
Markdownによる要件・設計管理

ドキュメントフォルダ構造を定義し、業務フローや仕様定義をMarkdown化。誰もが容易に編集・更新可能です。



Pull Requestでの直接レビュー

非エンジニアも成果物のファイルリストを見ながら、修正や特定行へのレビューコメントが可能。議論が一箇所に集約されます。



実例: docs/ フォルダ構成とREADMEによる
SSOT

sample-project

ガイドラインに準拠したトラッカー（Issue分類）とラベル運用

分類（Tracker）	GitHub ラベル	主な起票者	役割と自動化における重要性
Epic	type:epic	Product Owner	ビジネス上の目的を概説。タイムラインやロードマップでの長期管理の起点。
Story	type:story	Product Owner	ビジネスフロー、機能ステップ、画面仕様。受け入れ基準（AC）を含み、AI実装のインプットに直結。
Dev Task	type:devtask	PM / Team Lead	プログラミングタスク（Frontend / Backend / AI / Infrastructureなど対応ラベルを付与）。
QA Bug	type:qabug	QA Team	内部テスト中に発見された不具合。重篤度（severity:criticalなど）を紐付け自動集計。
Feedback	type:feedback	Product Owner	UAT（受入テスト）中の顧客フィードバックや不具合の一時プール。のちにBugやStoryへ変換。

GitHub Project v2 の基本管理規則

🗒️ 1. 事前設定された3つのView(タブ)運用

- **Table ビュー:** データー一括編集や高度なフィルタリング制御。
- **Board ビュー:** デイリースクラム用。ステータス間をドラッグ&ドロップで進捗管理。
- **Roadmap ビュー:** タイムラインに沿った長期のEpic・マイルストーン管理。

⚙️ 2. デフォルトカスタムフィールドの定義

正確な進捗測定（KPI Insightへの情報連携）のために必須化：

- **Status / Target Version / Priority** (Critical, High, Medium, Low)
- **Estimated time / Spent time** (計画時間 / 実実績時間)
- **Start Date / Due Date** (Roadmap上でロードマップを描画)

📌 3. チケット起票・命名規則

- **命名構文:** [Tracker] 簡潔なタスク名
- **(例):** [Dev Task] ログイン API を設計する
- **Assignee:** 少なくとも1名の主担当者を必ずアサインします。

🔗 4. PR（プルリクエスト）との自動紐付け

PR経由でIssueステータスを自動遷移させるため、PRのタイトルまたは説明文に必ず
Close #[ID Task]形式でIssue識別子を含めます。
これによりトレーサビリティデータが自動接続されます。

なぜGitHubを中心ツールにしたのか



業界の圧倒的な普及性

Sun*は多くのクライアントワークを支援しています。特殊な社内ツールに閉じるのではなく、グローバルスタンダードであり、多くの企業に既に導入・受け入れられているGitHubを型化して再利用することが最適でした。



情報の1ステップ集約

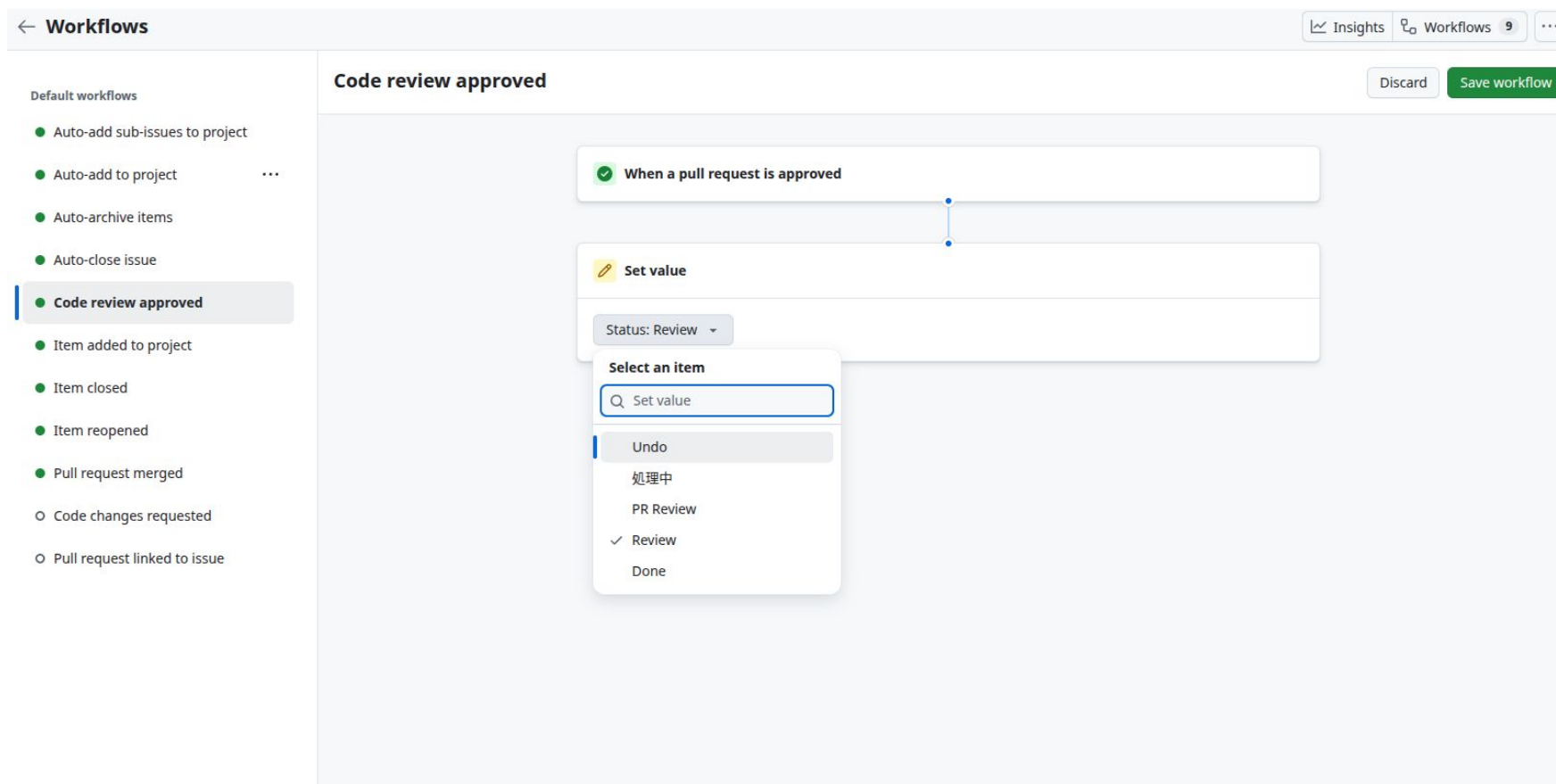
Issue・PR・コード・CI/CDログ・進捗管理までがすべて同一のエコシステム上に格納されています。ツールの行き来を無くし、コンテキストスイッチコストを極小化することが開発プロセスの摩擦を減らすために必要でした。



AI・自動化との高い親和性

GitHubは非常に強力なWeb APIやGitHub CLI、Actionsを提供しています。AIエージェントによるIssue読み取り、コードの自動生成、進捗データの自動抽出など、「AIとの対話」が最も円滑に行える共通プラットフォームです。

有用な設定が多くある



ソースのバージョン管理におけるアクションがそのままタスク管理にも反映されるなど

→一つのツールで複数の概念(タスク、ソースコード)を管理できる
→ Single Source Of Truthを達成しやすい

実際にどう運用しているか

STEP 01



基本設計Issue起票

システム全体の基本要件を「基本設計Issue」として定義。大まかなマイルストーンを決定します。

#Issue-010

STEP 02



詳細設計への分解

基本設計を「詳細設計Issue」へ細分化。AIコーディングツールに最適な独立したタスク単位にします。

#Issue-010-01

STEP 03



実装PRへ接続

AI開発者やエンジニアがブランチを作成しコーディング。PRの作成時に対応する詳細設計Issueを自動紐付け。

PR #204

STEP 04



CI/CD & マージ

PR作成後、Self-hosted Runnerによる自動ビルド/テストが起動。エンジニアレビューを経てマージへ。

🔗 この連続性（トレーサビリティ）が、高品質な自動化を支えるインフラです

（Issue番号とPR番号の相互関連付けにより、自動的に仕様書が同期されます）

過去レビューデータを活用したAIの継続的精度向上

AI駆動による業務フロー定義や設計、実装をさらに効率化するために重要なのは、過去の指摘データをAIのプロンプト（プロンプトチューニング）へ循環させる学習モデルです。



1. コメントデータの抽出

GitHub上で蓄積された「PR（Pull Request）での業務フローやコードへのレビュー指摘コメント」をAPI経由で自動吸い上げ。



2. プロンプトチューニング

過去の指摘（暗黙知、プロダクト固有の制約、要件フロー設計のパターンなど）を、AIエージェントのシステムインプットへ自動還元。



3. 成果物精度の永続的向上

同じ過ちや指摘を繰り返さない「プロダクト特化型のAI」へ自律進化。業務フローやコード生成の初期出力品質を極限まで高めます。

AI駆動開発における新たなマネジメント課題

🕒 従来の開発プロセス

PRの数が適正：開発チームのコード実装速度は人間の手による限界内にあり、全体のボリュームをPMが感覚的に把握可能。

変化が緩やか：日々の変更点が緩やかであり、レビューやマージのタイミングを個別に容易に追い切れる。

滞留に気づきやすい：進捗の遅れは、デイリーの口頭ミーティングやシンプルなボード上で容易に検知可能。

⚠️ AI駆動開発：爆発的な流量による難化

PR数が激増：AIの高いコード作成効率により、短期間に大量のIssue/PRが爆発的にリリースされます。

変化が超高速：ステータスの変化速度が速く、人間（マネージャー）が目で追い切る限界を容易に突破します。

滞留のブラックボックス化：どの詳細設計にボトルネックがあり、どのPRがレビュー待ちで滞留しているかが埋もれやすい。

→ スループットが上がるほど、可視化されていないマネジメントは不確実になり破綻します

解決策：KPI Insightの開発と導入

GitHub Projects とカスタムフィールド（Custom Fields）をコアエンジンとして活用し、開発フロー中の全ての進捗データをリアルタイムに自動集計・ダッシュボード化するSun*独自のマネジメントシステムです。

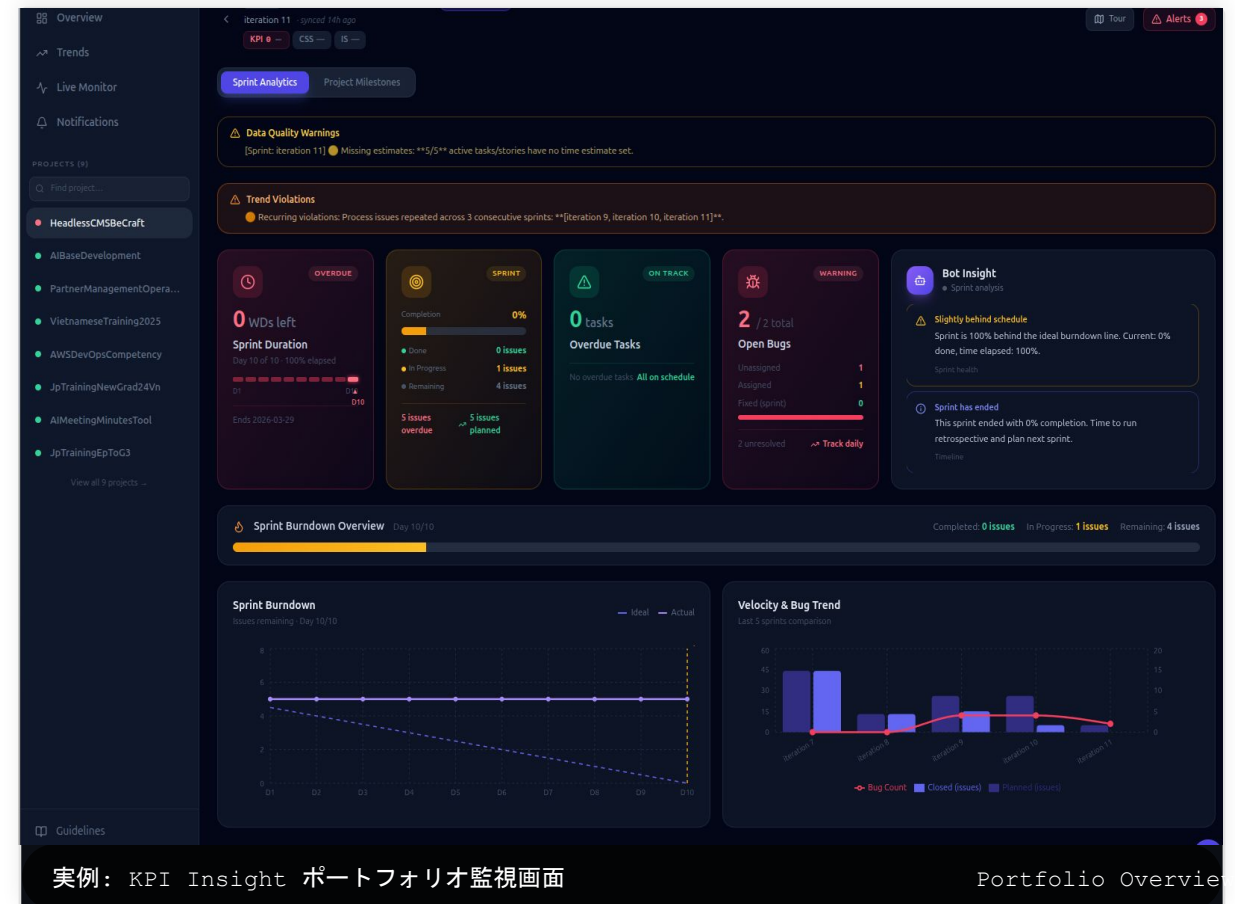
完全自動のデータ収集

GitHub Projectsからプロジェクト状況を同期。手動でのエクスポートや二重管理の手間をゼロに。

リアルタイム監視・異常検知

進捗遅延（Overdue）や品質、スプリントの進捗、バグ数などを統合し、「At Risk」や「Critical」を自律判定。

※APIとしてGituHub上から提供される情報の粒度が多いからこそ実現できた



KPI Insightで見える化している主要な指標

1. 量 (Volume) を見る

プロジェクトの規模、開発チームの活動度合いを数値から把握します。

Issue 総起票数：基本設計・詳細設計の分解数。

PR (Pull Request) 処理数：AI・人間が処理したPR数。

作成コード総量：開発が順調に展開しているかのインディケータ。

2. 流れ (Flow) を見る

開発ライフサイクル内の各ステージが、どれくらい速く通過しているか。

リードタイム (Lead Time)：タスク起票から最終マージ・完了までの総所要時間。

フェーズ別遷移スピード：「要件定義」から「詳細設計」、さらに「実装」まで、各プロセス間を流れる平均時間。

3. 詰まり (Stall) を見る

滞留（ボトルネック）が発生している、今すぐ介入すべき箇所を早期発見。

フェーズ別滞留件数：特定フェーズに設定上限を超えて留まるIssue数。

レビュー待ち滞留時間：PR作成からエンジニアがレビューを開始するまでの経過時間。

可視化で変わった運営・意思決定

× Before（導入前の課題）

感覚ベースの進捗確認：PMが「開発はおそらく順調だろう」という感覚値でミーティングや意思決定をせざるを得なかった。

遅れてから気づくボトルネック：週次の会議で「実はレビューが3日間滞留していた」と発覚し、軌道修正が遅れる。

属人的な対応：特定のリーダーだけが進捗把握に奔走。

✓ After（導入後の成果・変化）

正確な数字ベースの会話：「現在のフェーズ滞留が平均36時間を超えている」など、ファクトから全員で会話ができる。

詰まりの早期発見と即時介入：アラートが表示された時点で即時にタスクアサインを調整し、1日未満でボトルネックを解消。

AI駆動プロセスの最適化：AIが生成するスループットと人間がレビューできるキャパシティの不均衡を早期に是正。

📌 意思決定のスピードと確実性が、AI駆動プロセスの真のボトルネックになります

まとめ：GitHubはAI駆動開発の共通基盤になる

01

全開発ライフサイクルを包括するGitHub

単なるソースコード置き場を超え、要件から詳細設計Issue、開発PR、デプロイにわたる全工程の中心として、一元的な情報を集約。

02

高速化に追従するマネジメント：KPI Insight

AIでの開発スピード加速が生む「情報の爆発的増加」をコントロールするため、正確なメトリクスの自動可視化が不可欠。

03

AI時代における「真の開発生産性」

AI時代のボトルネックは実装力ではなく、それを支える設計・承認・可視化の仕組みにあります。Sun*はこれからもこれをリードします。